

이번 과제에서 2D 리스트(2D List) 구조를 쓰려고 해. 식단 리스트에 음식 이름이랑 칼로리를 한 쌍으로 묶어서 append로 누적 추가하는 방법을 알려줘. 그리고 이걸 그냥 출력하면

질문 잘 주셨습니다! 4차 과제의 핵심인 이중 리스트(2D List)와 인덱싱 표 출력을 상우님이 기존에 작성하신 코드(V3.0) 구조에 자연스럽게 녹여낼 수 있도록 조언과 함께 코드 틀을 짜 드릴게요.

보고서에 그대로 넣기 좋게 교수님 제출용 대학생 스타일의 답변으로 정리했습니다.

🤖 AI 파트너의 답변

기존 3차 과제에서는 `daily_meals = ["닭가슴살", "사과"]` 같은 1차원 리스트 형태여서 하나의 정보(음식명)만 담을 수 있었습니다.

이번 4차 과제에서는 여러 항목을 동시에 다루기 위해, 리스트 안의 각 방에 `[음식명, 칼로리]` 라는 또 다른 리스트 세트를 통째로 집어넣는 이중 리스트(2D List) 구조로 변경해야 합니다.

1. `append` 를 활용한 이중 리스트 누적 및 추가 방법

데이터를 넣을 때 음식명 문자열과 칼로리 숫자를 하나의 대괄호(`[]`)로 묶어서 `append` 해주면 됩니다.



python



```
# 기존 3차 방식: 한 가지만 추가 가능
# daily_meals.append(meal_input)

# 변경할 4차 방식: [음식명, 칼로리] 세트를 묶어서 append (이중 리스트 구조)
daily_meals.append([meal_input, calories])
```

이렇게 하면 메모리상에는 `[["닭가슴살", 120], ["사과", 150]]` 형태로 차곡차곡 데이터가 누적됩니다.

2. `for` 문과 리스트 인덱싱을 활용한 표 포맷 출력 코드 틀

이중 리스트를 단순 `print(daily_meals)` 로 출력하면 대괄호가 겹쳐 나와 가독성이 떨어집니다.

`for` 문과 `range()` 를 쓰고, 행과 열을 지정하는 **인덱싱(`[i][0]` , `[i][1]`)**을 통해 깔끔한 표 모양으로 뽑아내야 합니다.

python



```
# 4번 메뉴(조희)에 들어갈 표 출력 코드 틀입니다.
print("\n" + "-" * 10 + " 현재까지 누적된 식단 표 " + "-" * 10)
```

```

print("번호 | 음식명 | 칼로리")
print("-" * 36)

# len(daily_meals)를 통해 저장된 총 식단 개수만큼 반복합니다.
for i in range(len(daily_meals)):
    # daily_meals[i][0] -> i번째 식단의 '음식 이름'
    # daily_meals[i][1] -> i번째 식단의 '칼로리'
    name = daily_meals[i][0]
    kcal = daily_meals[i][1]

    # 열 맞춤용 포맷 출력
    print(f" {i+1} | {name} | {kcal} kcal")

print("-" * 36)

```

💡 상우님이 작성하신 코드에서 고치면 좋은 점 (조언)

1. **식단 추가(3번 메뉴) 수정:** 음식을 입력받는 `input()` 밑에 칼로리를 입력받는 `calories = int(input(...))` 코드를 추가하고, 저장할 때 반드시 `[meal_input, calories]` 형태로 묶어서 `append` 하도록 위치를 수정해 주세요.
2. **총 칼로리 누적 연산 추가:** 표를 출력하는 `for` 루프 내부에서 `total_calories += daily_meals[i][1]` 같은 누적 연산자(`+=`)를 함께 배치하면, 표 출력과 동시에 오늘 먹은 총칼로리 합산까지 한 번에 부드럽게 해결할 수 있어서 코드가 훨씬 깔끔해집니다.

다음 단계인 `with open`을 활용한 파일 저장과 `try-except` 예외처리 최소 2곳 이상 적용**에 관한

입력이랑 표 출력은 이해했어. 그럼 이제 이
누적된 이중 리스트 데이터를 프로그램이 꺼지기
전에 `with open()`을 써서 메모장 파일(.txt)로
정상 저장(Write)하는 함수를 만들고 싶어.
그리고 사용자가 칼로리를 넣을 때 문자를
입력하거나 메뉴를 잘못 골라도 프로그램이
비정상 종료되지 않게 `try-except` 예외 처리를
최소 2곳 이상 적용하고 싶은데 코드를 어떻게
수정해야 할까

아주 훌륭한 질문입니다! 프로그램의 안정성을 높이는 `try-except` 예외 처리와 데이터를 영구적으로 보관하는 `파일 입출력(with open)**`은 실제 개발에서도 가장 중요하게 다루는 항목입니다.

이 두 가지 필수 조건을 상우님의 V4.0 코드 구조에 대학생이 직관적으로 이해하고 짠 듯한 정석적인 코드로 반영하는 방법과 조언을 정리해 드립니다. 보고서에 그대로 활용하기 좋습니다.

🤖 AI 파트너의 답변

1. `with open()` 을 사용한 이중 리스트 데이터 파일 저장 (Write)

프로그램이 꺼지기 전(5번 종료 메뉴)에 현재까지 입력된 `daily_meals` 이중 리스트를 텍스트 파일(.txt)로 저장하는 독립된 함수를 만듭니

이 함수는 파일 쓰기 함수인 `with open`을 사용하여 데이터를 파일에 저장하는 역할을 합니다.

1. with open() 을 사용한 이중 리스트 데이터 파일 저장 (Write)

프로그램이 꺼지기 전(5번 종료 메뉴)에 현재까지 누적된 `daily_meals` 이중 리스트를 텍스트 파일(`.txt`)로 저장하는 독립된 함수를 만듭니다.

`with open()` 구문은 파일 쓰기가 끝나면 자동으로 파일을 닫아주므로 메모리 누수를 막아주는 안전한 슬라이드 예제 스타일 문법입니다.

python

```
# 5번 메뉴 선택 시 백업용 파일 저장을 담당하는 함수
def file_save_func():
    # 'w' 모드로 열어서 diet_records.txt 파일을 생성하거나 덮어씁니다.
    with open("diet_records.txt", "w", encoding="utf-8") as file:
        file.write("=== 스마트 피트니스 데이터 백업 ===\n")
        file.write("[이중 리스트 식단 데이터 목록]\n")

    # for문을 돌면서 이중 리스트 내부의 데이터를 한 줄씩 꺼내 파일에 씁니다.
    for m in daily_meals:
        # m[0]은 음식명, m[1]은 칼로리 숫자 데이터
        file.write(f"- {m[0]} : {m[1]}kcal\n")

    print("[파일 저장 완] 'diet_records.txt' 파일에 데이터가 정상 저장되었습니다.")
```

2. try-except 예외 처리 구문 적용 (최소 2곳 이상 필수 조건 충족)

사용자가 잘못된 입력값(텍스트, 특수문자 등)을 넣었을 때 빨간색 에러 메시지와 함께 프로그램이 강제로 튕기는(비정상 종료) 현상을 완벽히 방어하기 위해 가장 취약한 2곳에 예외 처리를 배치합니다.

• [1곳째] 3번 식단 추가 함수 (칼로리 숫자가 아닌 문자 입력 방지):

사용자가 칼로리 입력창에 삼백이나 300kcal 처럼 문자를 섞어 넣으면 `int()` 형변환 과정에서 `ValueError` 가 발생합니다. 이를 잡아내어 프로그램이 꺼지지 않게 막아줍니다.

python

```
try:
    # 문자가 들어오면 에러가 발생하는 위험 지역
    kcal = int(input("해당 음식의 칼로리(kcal)를 입력하세요: "))
except ValueError:
    # 에러 발생 시 튕기지 않고 실행되는 방어 코드
    print("[오류] 칼로리는 숫자만 넣어야 합니다! 0kcal로 임시 저장합니다.")
    kcal = 0
```

• [2곳째] 하단 메인 루프 메뉴 선택부 (메뉴 오입력 및 예기치 못한 에러 방지):

`while True` 루프 내부에서 메뉴 번호를 선택하고 입력받는 흐름 전체를 `try-except` 로 감싸주면, 사용자가 어떤 기상천외한 값을 입력하거나 시스템 내부 오류가 발생하더라도 루프가

- [2곳째] 하단 메인 루프 메뉴 선택부 (메뉴 오입력 및 예기치 못한 에러 방지):

`while True` 루프 내부에서 메뉴 번호를 선택하고 입력받는 흐름 전체를 `try-except` 로 감싸주면, 사용자가 어떤 기상천외한 값을 입력하거나 시스템 내부 오류가 발생하더라도 루프가 깨지지 않고 유지됩니다.

python



```
while True:
    # ... 메뉴판 출력 생략 ...
    try:
        user_choice = input("원하는 기능의 번호를 선택하세요: ")

        if user_choice == "1":
            regist_info()
        # ... 종락 (2번, 3번, 4번 메뉴 분기) ...
        elif user_choice == "5":
            file_save_func() # 종료 전 파일 저장 함수 호출
            print("\n프로그램을 종료합니다.")
            break # break로 무한 루프 탈출

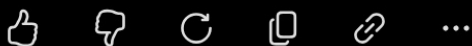
    except Exception as error_msg:
        # 예기치 못한 모든 에러를 포괄적으로 잡아서 안내 메시지만 띄움
        print(f"\n[오류 발생] 알 수 없는 에러가 발생했습니다: {error_msg}")
```

💡 상우님이 작성하신 코드에서 고치면 좋은 점 (조언)

1. 함수 호출 위치 제어: 5번 종료 메뉴 분기(`elif user_choice == "5":`)가 실행되었을 때, 곧바로 `break` 를 해서 프로그램을 종료하기 전에 반드시 `file_save_func()` 를 먼저 호출하도록 순서를 배치해야 데이터가 유실되지 않고 안전하게 저장됩니다.
2. `ValueError` 예외 처리 시 대안 설계: 예외 처리를 할 때 그냥 통과시키는 것보다는 `kcal = 0` 처럼 변수에 기본값을 강제로 대입해 주는 방어 코드를 짜두어야, 뒤이어 나오는 4번 메뉴(`show_result`)에서 이중 리스트 연산을 할 때 변수 미선언 에러(`NameError`)가 연쇄적으로 터지는 것을 막을 수 있습니다.

이렇게 해서 4차 과제의 모든 필수 조건(이중 리스트, 파일 저장, 예외 처리 2곳)을 완벽하게 관통하는 설계를 마쳤습니다!

이 내용을 기반으로 타이핑을 해보시면서 과제 원본 소스코드를 완성해 보세요. 혹시 코드가 막히거나 보고서 내용에 대해 더 질문하고 싶으신 점이 있다면 편하게 말씀해 주세요!



Gemini는 AI이며 실수를 할 수 있습니다.

